

Concurrencia POSIX

Sincronización

Antonio Espín Herranz

Contenidos

- Hilos POSIX
- Sincronización
- Problemas clásicos

HILOS: Conceptos

- **Proceso:** podemos considerar un proceso como un programa en ejecución.
 - Un mismo programa puede derivar en varios procesos.
 - Tener varios ejemplares en ejecución de un programa, por ejemplo, Word o cualquier otro.
 - Y también un mismo programa puede necesitar utilizar diferentes procesos:
 - Por ejemplo, Word puede requerir utilizar un editor de ecuaciones, o de gráficos.

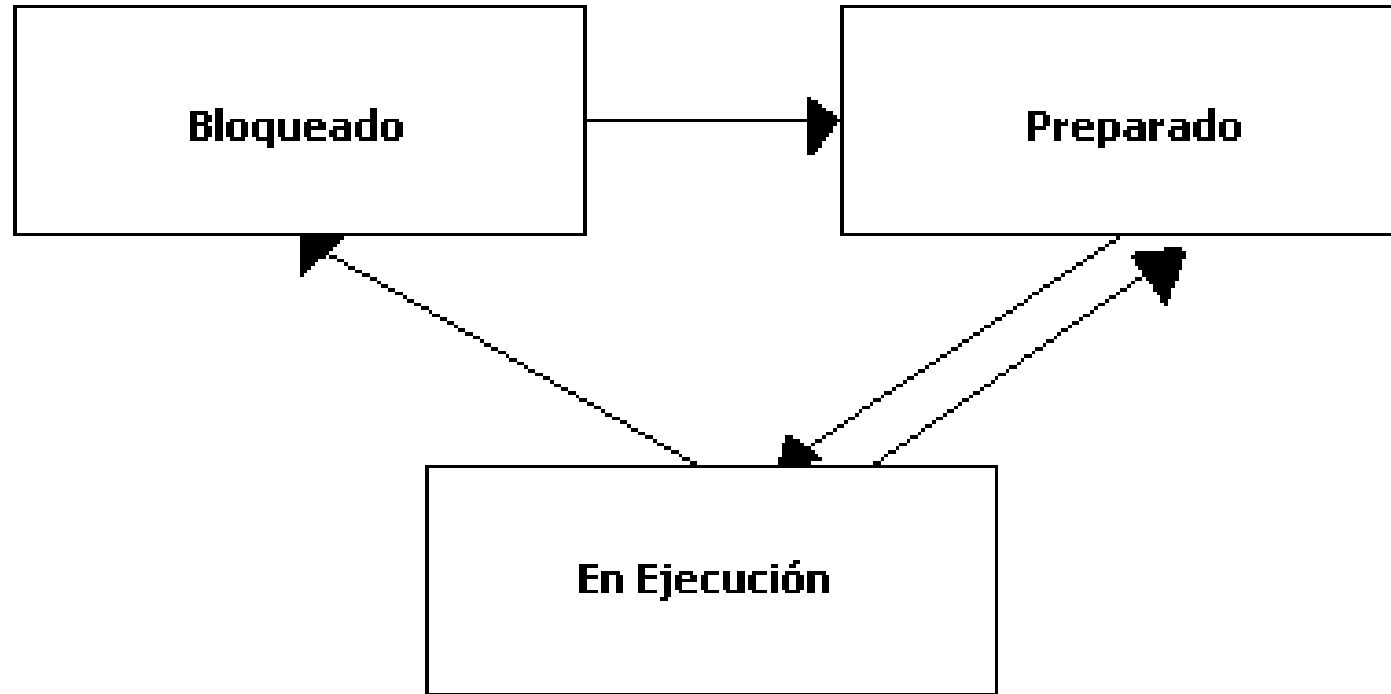
Procesos

- Cada proceso consta de bloques de código y de datos cargados desde un fichero ejecutable.
- Además, un proceso posee:
 - Su propio espacio de direcciones.
 - Sus variables.
 - Ficheros abiertos.
 - Referencias a procesos hijos.
 - Registros, pila, etc.
- El equivalente a una CPU virtual.
- Los procesos se controlan por el Sist. Oper. Y se van alternando en la utilización de la CPU.

Estados de un proceso

- Posibles estados de un proceso:
 - **Ejecución:** está utilizando CPU.
 - **Preparado:** está detenido temporalmente para que se ejecute otro proceso.
 - **Bloqueado:** El proceso está esperando a que ocurra algo para continuar. Puede estar esperando leer datos de la entrada estándar.

Ciclo de vida del proceso

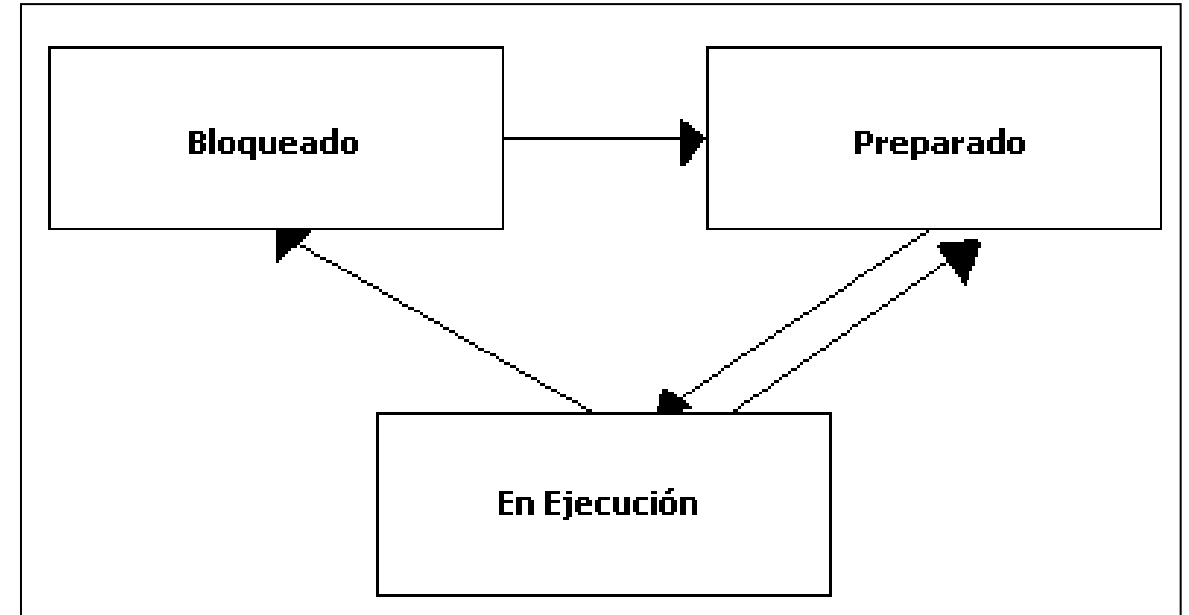


Hilos

- Un hilo (***thread***), proceso ligero.
- Es la unidad de ejecución de un proceso.
- Dispone de:
 - Secuencia de instrucciones.
 - De un conjunto de registros.
 - Y una pila.
- Cuando se crea un proceso, el Sist. Ope. Crea un primer hilo (hilo primario), que a su vez puede crear hilos adicionales.
- Un proceso puede ejecutar de forma concurrente varias secuencias de instrucciones.
- Cada secuencia de instrucciones recibe el nombre de hilo (thread), todos los hilos comparten el espacio de memoria virtual del proceso y los recursos asignados por el sistema operativo.

- **Nuevo:** El hilo se ha creado, pero no se ha activado. De aquí pasará a preparado.
- **Preparado:** Está activo y a la espera de que se le asigne la CPU.
- **Ejecución:** Está activo y se le ha asignado la CPU.
- **Bloqueado:** El hilo espera a que termine otro hilo. Puede estar:
 - **Dormido:** Se bloquea durante x tiempo, de aquí pasará al estado de preparado.
 - **Esperando:** El hilo está esperando a que ocurra alguna cosa: espera una operación de E/S o adquirir una propiedad de un método sincronizado.
- **Muerto:** El hilo ha terminado y no ha sido recogido por el padre. De aquí ya no puede ir a ningún estado.

Estados de un hilo



Biblioteca POSIX

- **Posix:** *Portable Operating System Interface*.
- Esta biblioteca se encuentra definida en el fichero **<pthread.h>**
- **Funciones:**
 - pthread_create: Crear hilos.
 - pthread_once: Ejecutar el código una vez.
 - pthread_join: Esperar a otro hilo.
 - pthread_equal: Comparar hilos.
 - pthread_detach: Alternativa a pthread_join.
 - pthread_self: Devuelve el identificador del hilo que invoca a esta función.

pthread_create

- **Recibe 4 parámetros:**

pthread_t *tid:

El identificador del hilo.

const pthread_attr_t *attr:

Se suele dejar a 0 para que utilice los parámetros por defecto de tamaño de pila.

void *(*start)(void *):

Puntero a una función. La que va a ejecutar el hilo. A esta función SOLO se le puede pasar un dato y devuelve un dato, en caso de necesitar mandar mas información, utilizar estructuras.

void *arg:

Es la dirección del argumento que se le pasará a la función start

Devuelve 0 si todo OK, si no un código de error.

pthread_once

- Recibe dos parámetros:

pthread_once_t *once_control:

- Nos asegura que el código que indiquemos en la rutina que vamos a pasar sólo se ejecutará una vez. El primer parámetro apunta a una variable static o extern iniciada a la constante: PTHREAD_ONCE_INIT.

void (*init_routine)(void):

- La dirección de la rutina a ejecutar.

Devuelve 0 si todo OK, si no un código de error.

pthread_join

- Recibe dos parámetros:

pthread_t thread: Un hilo.

- Esta función suspende la ejecución del hilo que la invoca, hasta que el hilo indicado por thread no termine su ejecución.

void **value_ptr

- El segundo parámetro cuando NO es cero, hace referencia al valor devuelto por el hilo identificado por thread (valor devuelto por return). Podemos poner 0.

- Esta función devuelve 0, si todo OK, sino != 0.

pthread_equal

- Recibe los dos identificadores de los hilos.
- De tipo pthread.
- Devuelve 0 si son distintos y si no devuelve != 0.

pthread_detach

- `int pthread_detach(pthread_t thread);`
 - Esta función es una alternativa de `join`.
 - Recibe el hilo: parámetro de tipo `pthread_t`.
- Un hilo detached:
 - continúa ejecutándose normalmente;
 - libera automáticamente sus recursos al terminar;
 - no necesita `pthread_join()`;
 - ya no puede recogerse con `pthread_join()`;
 - sigue perteneciendo al mismo proceso

pthread_detach

- Diferencias con pthread_join:

Acción	Efecto
pthread_join()	Espera al hilo y recupera sus recursos
pthread_detach()	Sus recursos se liberan automáticamente al finalizar
return desde main()	Termina el proceso y todos sus hilos
exit()	Termina el proceso y todos sus hilos
pthread_exit() en main()	Termina solo el hilo principal

pthread_detach

- Si main() retorna o llama a exit(), termina el proceso completo, incluidos todos los hilos detached que sigan ejecutándose. pthread_detach() no evita esto.
- pthread_detach() suele usarse para hilos de tipo “**lanzar y olvidar**”, cuyo resultado no necesita recoger ningún otro hilo.

pthread_detach

- **Usos habituales:**
 - Atender clientes en un servidor con un hilo por conexión.
 - Ejecutar tareas auxiliares independientes.
 - Procesar trabajos en segundo plano.
 - Registrar eventos o métricas.
 - Ejecutar una tarea corta sin devolver resultado.

pthread_detach

No conviene usarlo cuando necesites:

- esperar a que termine el hilo
 - recoger su valor de retorno
 - conocer exactamente cuándo finalizó
 - coordinar el cierre ordenado del programa.
-
- En esas situaciones, utilizar join:
 - `pthread_join(thread, NULL);`

pthread_self

- Devuelve el identificador del hilo que la invoca.
- No tiene parámetros.

Creación / Destrucción de Hilos

- Incluir el fichero de cabecera:
`#include <pthread.h>`
- Escribir las instrucciones del hilo dentro de una función, con el prototipo:
`void *fnHilo(void *p);` // Al ser punteros genéricos podemos pasar y devolver cualquier cosa.
- Definir el identificador del hilo(s):
`pthread_t id_hilo;`
- Crear el hilo e iniciar su ejecución:
`pthread_create(&id_hilo, 0, fnHilo, 0);`
- Desde el hilo primario esperamos a que termine el hilo secundario.
`pthread_join(id_hilo, 0);`

Ejemplo

```
#include <stdio.h>
#include <pthread.h>
#include <Windows.h>

int random(int n) {
    static int primera_vez = 1;
    if (primera_vez) {
        srand(time(0));
        primera_vez = 0;
    }
    return (rand() % n) + 1; // valor entre 1 y n
}
```

Ejemplo (2)

```
void *tomarMuestraTipoA(int *n) {  
    *n = random(10);  
  
    printf("Vamos a tomar %d muestras ...\n" , *n );  
    for (int i = 0; i < *n; i++) {  
        printf ("Tomando muestra de tipo A\n");  
    }  
    Sleep(2);  
    return n;  
}
```

```
void *tomarMuestraTipoB(int *n) {  
    *n = random(15);  
  
    printf("Vamos a tomar %d muestras ...\n" , *n );  
    for (int i = 0; i < *n; i++) {  
        printf ("Tomando muestra de tipo B\n");  
    }  
    Sleep(5);  
    return n;  
}
```

**FUNCIONES QUE SE VAN A
EJECUTAR CON HILOS**

**TIENE QUE SER DEL
PROTOTIPO**

void *funcion(void *)

**Punteros genéricos →
conversiones implícitas.**

Ejemplo (3)

```
void resultados(int n, int m) {  
    printf("Muestras de tipo A: %d \n", n);  
    printf("Muestras de tipo B: %d \n", m);  
    printf("Total: %d\n", n + m );  
}
```

```
typedef void *(*ptrfnthr)(void *);  
int main() {  
    int nMuestrasTipoA = 0;  
    int nMuestrasTipoB = 0;  
    int codigo, *estado;  
  
    // Identificadores de los hilos  
    pthread_t hilo1, hilo2;  
    // Crear los hilos 1 y 2  
  
    printf( "Creamos el hilo 1\n");  
    pthread_create(&hilo1, 0, (ptrfnthr) tomarMuestraTipoA, (void *) &nMuestrasTipoA);  
  
    printf("Creamos el hilo 2\n");  
    pthread_create(&hilo2, 0, (ptrfnthr) tomarMuestraTipoB, (void *) &nMuestrasTipoB);  
  
    // Esperar a que los hilos terminen  
    codigo = pthread_join(hilo1, (void **) &estado );  
    codigo = pthread_join(hilo2, (void **) &estado);  
  
    // Mostrar resultados  
    resultados(nMuestrasTipoA, nMuestrasTipoB);  
  
}
```

Paso de parámetros (1 solo parámetro)

```
#include <pthread.h>
#include <stdio.h>

static void *worker(void *argument){
    int value = *(int *)argument;
    printf("Valor recibido: %d\n", value);
    return NULL;
}
```

```
int main(void){
    pthread_t thread;
    int value = 42;
    int error = pthread_create( &thread,
                                NULL, worker, &value );

    if (error != 0) {
        return 1;
    }
    pthread_join(thread, NULL);
    return 0;
}
```

Paso de parámetros (N parámetros)

- Si necesitamos pasar mas de un parámetro a nuestro hilo tendremos que definir una estructura.
- Se definirá de la siguiente manera:

```
typedef struct {  
    tipo var;  
    tipo2 var2;  
} pthread_param_t;
```

Paso de parámetros (N parámetros)

```
#include <pthread.h>
#include <stdio.h>
```

```
typedef struct {
    int thread_id;
    int start;
    int end;
    const char *name;
} thread_args_t;
```

```
static void *worker(void *argument)
{
    thread_args_t *args = argument;
    printf(
        "Hilo %d (%s): rango %d-%d\n",
        args->thread_id,
        args->name,
        args->start,
        args->end
    );
    return NULL;
}
```

```
int main(void){
    pthread_t thread;

    thread_args_t args = {
        .thread_id = 1,
        .start = 0,
        .end = 100,
        .name = "worker-1"
    };

    if (pthread_create( &thread, NULL, worker, &args) != 0) {
        return 1;
    }

    pthread_join(thread, NULL);
    return 0;
}
```

Finalizar Hilos

- Tenemos varias formas de terminar un hilo:
 - **Retornando** de la función de ejecución.
 - `return ...`
 - Llamando él mismo a **pthread_exit**.
 - `void pthread_exit(void *);`
 - Invocando a la función **pthread_cancel** pasando por argumento su identificador.
 - `pthread_cancel(pthread_t);`
 - *Esta es delicada, podemos cancelar un hilo que está interactuando con un recurso y no lo ha liberado.*

Ejemplo

```
void *tomarMuestra(pthread_param_t *p) {  
    p->cuenta = random(15);  
    for (int i = 0; i < p->cuenta; i++)  
        printf("%s\n", p->str);  
    return &p->id;  
}
```

```
void tomarMuestra(pthread_param_t *p) {  
    p->cuenta = random(15);  
    for (int i = 0; i < p->cuenta; i++)  
        printf("%s\n", p->str);  
    pthread_exit(&p->id);  
}
```

Finalizar hilos

- Finalizar main, pero que el resto de hilo continúen funcionando:

```
int main(void){  
    pthread_t thread;  
    pthread_create(&thread, NULL, worker, NULL);  
    pthread_detach(thread);  
    pthread_exit(NULL); // Termina el hilo que lo ejecuta  
}
```

Finalizar hilos

- Pero si ejecutamos **return 0** o **exit(EXIT_SUCCESS)** desde **main**, termina todo el proceso con sus hilos.
- Si queremos que una tarea sobreviva al proceso original entonces **no debe ser un hilo**, debe ser un **proceso** (se crea con **fork()**)

Número de hilos en funcionamiento

- Número de **hilos existentes**
- vs
- Número de **hilos ejecutándose en paralelo** depende del número de núcleos del procesador.
 - Dos hilos por cada núcleo.
- Obtener el número de procesadores lógicos (2 por núcleo)
`#include <unistd.h>`
`long logical_cpus = sysconf(_SC_NPROCESSORS_ONLN);`
 - Desde una terminal: **nproc**

Número de hilos en funcionamiento

- **Hilos existentes:**
- Podemos crear muchos más hilos que núcleos. El límite depende principalmente de:
 - memoria disponible;
 - tamaño de la pila de cada hilo;
 - límites del usuario y del sistema;
 - recursos internos del kernel.
- **pthread_create()** devuelve **EAGAIN** cuando faltan recursos o se alcanza algún límite del sistema. Cada hilo tiene su propia pila
- **Hilos ejecutándose en paralelo:**
 - 4 núcleos físicos
 - 2 hilos hardware por núcleo
 - 8 procesadores lógicos
 - El sistema puede ejecutar aproximadamente ocho hilos simultáneamente.
 - Si tenemos 100 hilos preparados, el planificador los alternará mediante cambios de contexto

Sincronización

- Habrá veces que nos interese trabajar con hilos independientes, es decir, no interfieren entre sí en la ejecución.
- Y otras veces, tendremos un esquema de colaboración, pueden acceder a una zona de datos / recursos compartidos → **sincronización.**
- Las zonas comunes donde acceden los hilos reciben el nombre de **secciones críticas.**

Sincronización (2)

- Dentro de la librería de Posix tenemos una serie de elementos de sincronización como son:
 - Secciones mutuamente excluyentes (los **mutex** o **semáforos binarios**).
 - Los mutex se utilizan para el acceso mutuamente exclusivo a recursos compartidos. Similares a los semáforos, pero el hilo que bloquea el mutex (el propietario) debe ser el mismo que lo libere.

Secciones críticas

- Una sección crítica para nosotros va a ser una zona de memoria compartida por dos o mas hilos.
- Podemos tener un array de muestras para almacenar datos y varios hilos que acceden simultáneamente a dicho array.
- La estructura podría ser algo así:

```
struct {  
    int muestras[MAX_MUESTRAS];  
    int ind;  
} st_bufer;
```

Secciones críticas (2)

- La función a ejecutar por los hilos sería algo así:

```
void *adquirirDatos(void *pid){
    int x = 0, id = *((int *)pid);
    do {
        if (st_bufer.ind >= MAX_MUESTRAS)
            return pid;
        x = random();
        printf( "hilo-%d tomo la muestra %d\n", id,st_bufer.ind );
        st_bufer.ind++;
    } while (st_bufer.ind < MAX_MUESTRAS);
    return 0;
}
```

Secciones críticas (3)

- El código tal cual está puede generar resultados inesperados, si lo ejecutamos con mas de un hilo se pueden duplicar valores o saltarse algún valor.
- Este problema se conoce con el nombre de condiciones de carrera.
- La sección de código en la que se actualizan datos comunes a mas de un hilo recibe el nombre de sección crítica.
- La forma de evitar problemas planteados es que cuando un hilo esté ejecutando código de su sección crítica, ningún otro hilo pueda hacerlo.
 - Este mecanismo se conoce como **Exclusión Mutua**.

Exclusión mutua

- La biblioteca Posix proporciona un objeto llamado **mutex** (exclusión mutua), recibe el nombre de **semáforo binario** o **cerrojo**.
- Con un mutex se pueden realizar dos tipos de operaciones:
 - Echar el cerrojo.
 - Sección crítica.
 - Quitar el cerrojo.
- Cuando un hilo adquiere el mutex tiene la propiedad de que cuando es adquirido por un hilo, ningún otro hilo puede adquirirlo hasta que sea liberado.

Mutex

- Los mutex se definen **a nivel global** o se pasa por **parámetro a los hilos**, pero todos los hilos deben ver el mismo mutex.
 - `pthread_mutex_t objeto_mutex;`
 - Las funciones que ofrece la librería para trabajar con mutex devuelve un 0 si va todo bien y algo distinto de 0 si hay error.

Funciones para trabajar con mutex

- `int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);`
 - Es la primera que se invoca para invocar al mutex.
 - Primer parametro el mutex, y el segundo los parámetros del mutex, con 0 se utilizan los valores por defecto.
- `int pthread_mutex_lock(pthread_mutex_t *mutex);`
 - Cuando se ha iniciado el mutex, se puede adquirir con un hilo para echar el cerrojo a su sección crítica.

Funciones para trabajar con mutex (2)

- `int pthread_mutex_unlock(pthread_mutex_t *mutex);`
 - Un hilo puede quitar el cerrojo que puso a su sección crítica.
- `int pthread_mutex_destroy(pthread_mutex_t *mutex);`
 - Destruye el mutex especificado, liberando todos los recursos adquiridos por el mismo.

Mutex Global

```
#include <pthread.h>
```

```
static int counter = 0;
```

```
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // Inicializa con los valores predeterminados
```

```
static void *worker(void *argument)
```

```
{
```

```
    (void)argument; // Para indicar que el parámetro no se utiliza!
```

```
    pthread_mutex_lock(&mutex);
```

```
    counter++;
```

```
    pthread_mutex_unlock(&mutex);
```

```
    return NULL;
```

```
}
```

- Mutex **global** con atributos predeterminados: **PTHREAD_MUTEX_INITIALIZER**.
- Mutex dentro de una **estructura** o con atributos especiales: **pthread_mutex_init()**.

Mutex estructura compartida

```
typedef struct {  
    int counter;  
    pthread_mutex_t mutex;  
} shared_data_t;  
  
shared_data_t shared;  
shared.counter = 0;  
if (pthread_mutex_init(&shared.mutex, NULL) != 0)  
{  
    return 1;  
}
```

```
static void *worker(void *argument) {  
    shared_data_t *shared = argument;  
    pthread_mutex_lock(&shared->mutex);  
    shared->counter++;  
    pthread_mutex_unlock(&shared->mutex);  
    return NULL;  
}  
  
pthread_create( &threads[i], NULL, worker,    &shared);  
  
// Al finalizar todos los hilos:  
pthread_mutex_destroy(&shared.mutex);
```

Codificación

- Dentro de la estructura, añadimos el mutex:

```
struct {  
    int muestras[MAX_MUESTRAS];  
    int ind;  
    pthread_mutex_t mutex_adquirir;  
} st_bufer;
```

Codificación (2)

- La función que ejecutan los hilos:

```
void *adquirirDatos(void *pid){
    int x = 0, id = *((int *)pid);
    do {
        cod = pthread_mutex_lock(&st_bufer.mutex_adquirir);
        if (st_bufer.ind >= MAX_MUESTRAS)
            return pid;
        x = random();
        printf("hilo-%d tomo la muestra %d \n",id,st_bufer.ind);
        st_bufer.ind++;
        cod = pthread_mutex_unlock(&st_bufer.mutex_adquirir);
    } while (st_bufer.ind < MAX_MUESTRAS);
    return 0;
}
```

- Desde main() creamos el mutex:
 - **cod = pthread_mutex_init(&st_bufer.mutex_adquirir, 0);**

Semáforos

- Un semáforo puede controlar varios recursos accedidos por distintos hilos simultáneamente.
- Incluir el fichero: ***semaphore.h***
- Definir un semáforo:
 - **`sem_t`** semaforo;
- Una vez definido el semáforo puede ser manejado a través de una serie de funciones:
 - `sem_init`

Funciones de los semáforos

- `int sem_init(sem_t *sem, int pshared, unsigned int value);`
 - Inicia el semáforo al valor `value`. Si `pshared` es 0 no se puede compartir con otros procesos.
- `int sem_wait(sem_t *sem);`
 - Espera a que el contador del semáforo sea positivo. Esto es, si es 0, el hilo que hace la llamada espera (pasa al estado de bloqueado) hasta que el contador sea incrementado por una operación `sem_post`, o hasta que el hilo sea interrumpido, y si es positivo lo decrementa en uno y retorna.

Funciones de los semáforos (2)

- `int sem_post(sem_t *sem);`
 - Incrementa en 1 el contador del semáforo si no hay hilos bloqueados, si hubiera hilos esperando por el semáforo, el primero de ellos será despertado y pasa de bloqueado a preparado.
- `int sem_destroy(sem_t *sem);`
 - Elimina el semáforo referenciado por `sem`.

Condvars

- Variables de condición: **pthread_cond_t**
- Una **variable de condición** permite que un hilo espere eficientemente hasta que se cumpla una condición sobre datos compartidos.
 - **Mutex** → protege el estado compartido
 - **Condvar** → permite esperar y notificar cambios

Condvars

- La inicialización muy similar a los mutex:
- Inicialización:
 - `pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;`
 - `pthread_cond_t condition = PTHREAD_COND_INITIALIZER;`
- Inicialización dinámica:
 - `pthread_mutex_init(&mutex, NULL);`
 - `pthread_cond_init(&condition, NULL);`

Condvars

- **Al finalizar:**

```
pthread_cond_destroy(&condition);  
pthread_mutex_destroy(&mutex);
```

- **Espera:**

```
pthread_mutex_lock(&mutex);  
while (!condition_is_true) {  
    pthread_cond_wait(&condition, &mutex);  
}
```

/ Utilizar el recurso compartido */*

```
pthread_mutex_unlock(&mutex);
```

pthread_cond_wait()

realiza atómicamente estas operaciones:

1. Libera el mutex.
2. Bloquea el hilo.
3. Cuando el hilo despierta, vuelve a adquirir el mutex.
4. Retorna con el mutex bloqueado.

Condvars

- Señalización:
 - Despertar a un hilo:
 - `pthread_cond_signal(&condition);`
 - Despertar a todos los hilos:
 - `pthread_cond_broadcast(&condition);`

Patrón habitual:

```
pthread_mutex_lock(&mutex);  
condition_is_true = 1;  
pthread_cond_signal(&condition);  
pthread_mutex_unlock(&mutex);
```

Problema del productor / consumidor

- Es un problema muy típico.
- La idea es que tenemos un hilo que produce (lo que sea) y otro que consume (lo que genera el otro).
- El productor no puede generar nuevos datos si el almacén está lleno.
- Y el consumidor no puede capturar datos si no hay.
- Tiene que establecerse un sincronismo entre ambos hilos.

P / C – con mutex y condvars

- El productor introduce elementos en una cola y el consumidor los extrae:

```
#define CAPACITY 8
```

```
typedef struct {  
    int elements[CAPACITY];  
    size_t first;  
    size_t count;  
    pthread_mutex_t mutex;  
    pthread_cond_t not_empty;  
    pthread_cond_t not_full;  
} queue_t;
```

Producer

```
void queue_push(queue_t *queue, int value){  
    pthread_mutex_lock(&queue->mutex);  
  
    while (queue->count == CAPACITY) {  
        pthread_cond_wait( &queue->not_full, &queue->mutex );  
    }  
    size_t position = (queue->first + queue->count) % CAPACITY;  
    queue->elements[position] = value;  
    queue->count++;  
  
    pthread_cond_signal(&queue->not_empty);  
  
    pthread_mutex_unlock(&queue->mutex);  
}
```

Consumidor

```
int queue_pop(queue_t *queue){
    pthread_mutex_lock(&queue->mutex);

    while (queue->count == 0) {
        pthread_cond_wait( &queue->not_empty, &queue->mutex );
    }
    int value = queue->elements[queue->first];
    queue->first = (queue->first + 1) % CAPACITY;
    queue->count--;

    pthread_cond_signal(&queue->not_full);

    pthread_mutex_unlock(&queue->mutex);

    return value;
}
```

P / C - con mutex y semáforos

- Necesitamos dos hilos: uno hará de consumidor y otro de productor.
- Dos semáforos, uno controla que esté lleno el buffer y el otro que esté vacío.
- A parte de un mutex para controlar el acceso exclusivo.

Esquema

```
const int ELEMENTOS_BUFER = 16;  
const int TOTAL_DATOS = 512;
```

```
struct {  
    int bufer[ELEMENTOS_BUFER]; // matriz de datos  
    int ind_p, ind_c; // índices del productor y del consumidor  
    pthread_mutex_t exmut; // exclusión mutua  
    sem_t sem_vacios; // semáforo contador de elementos vacíos  
    sem_t sem_llenos; // semáforo contador de elementos llenos  
} st_bufer;
```

- El semáforo vacíos se inicializa al número de elementos del buffer.
- El otro a cero, al inicio no hemos puesto ninguno y están todos libres.

Productor

- Espera al semáforo vacíos.
- Echar el cerrojo.
- Genera el carácter y lo pone en el buffer.
- Trabaja con el índice del productor.
- Quita el bloqueo del cerrojo.
- Incrementa el semáforo de llenos.

Consumidor

- Espera al semáforo de llenos.
- Echa el cerrojo.
- Recupera un dato del buffer.
- Trabaja con el índice del consumidor.
- Desbloquea el cerrojo.
- Incrementa el semáforo de vacíos.

RWLocks

Un lock de lectura/escritura permite:

- varios lectores simultáneos;
 - un único escritor;
 - ningún lector mientras escribe un hilo.
-
- Definición: `pthread_rwlock_t`
 - Inicialización: `pthread_rwlock_t lock = PTHREAD_RWLOCK_INITIALIZER;`

RWLocks

- **Bloqueo para lectura:**

```
pthread_rwlock_rdlock(&lock);  
/* Consultar datos */  
pthread_rwlock_unlock(&lock);
```

- **Bloqueo para escritura:**

```
pthread_rwlock_wrlock(&lock);  
/* Modificar datos */  
pthread_rwlock_unlock(&lock);
```

- **Intentos sin bloqueo:**

```
pthread_rwlock_tryrdlock(&lock);  
pthread_rwlock_trywrlock(&lock);
```

RWLocks

Son adecuados cuando:

- las lecturas son mucho más frecuentes que las escrituras;
 - las secciones de lectura tienen suficiente duración;
 - permitir lectores simultáneos mejora realmente el rendimiento.
-
- No siempre superan a un mutex. Introducen más lógica interna y pueden sufrir inanición de lectores o escritores según la implementación y el patrón de acceso.

SpinLocks

- Un **spinlock** no duerme al hilo cuando está ocupado.
- El hilo permanece ejecutando un bucle hasta obtenerlo:
 - ¿**Está libre?**
 - **No** → volver a comprobar
 - **No** → volver a comprobar
 - **Sí** → adquirir

SpinLocks

- **Tipo:**
 - `pthread_spinlock_t`
- **Inicialización:**
 - `pthread_spinlock_t lock;`
 - `pthread_spin_init(&lock, PTHREAD_PROCESS_PRIVATE);`
- **Uso:**
 - `pthread_spin_lock(&lock);`
 - **`/* Sección crítica muy corta */`**
 - `pthread_spin_unlock(&lock);`
- **Destrucción:**
 - `pthread_spin_destroy(&lock);`

Si otro hilo posee el **spinlock**, **pthread_spin_lock()** permanece comprobándolo activamente hasta poder adquirirlo

Cuando se usa

- **Utilizarlo cuando:**

- Secciones críticas extremadamente cortas.
- Sistemas con varios procesadores
- El propietario liberará el Lock inmediatamente
- No se realizan acciones bloqueantes dentro de la sección crítica

- **NO utilizarlo cuando:**

- Tenemos sistemas con un único procesador lógico.
- Secciones críticas largas.
- Operaciones de red, disco o espera.
- Sobrecarga elevada del sistema
- Cuando el hilo propietario puede quedar desplanificado

Un mutex suele ser la opción predeterminada

Problemas típicos en tiempo real

- Inversión de la prioridad
- Secciones críticas demasiado largas
- Asignación dinámica
- Spinlocks y prioridad
- Planificación y permisos

Inversión de la prioridad

Se produce cuando:

- 1.Un hilo de baja prioridad adquiere un mutex.
- 2.Un hilo de alta prioridad intenta adquirirlo y queda bloqueado.
- 3.Un hilo de prioridad media ocupa la CPU.
- 4.El hilo de baja prioridad no puede ejecutarse para liberar el mutex.

Inversión de la prioridad

- Prioridad alta → espera el mutex
- Prioridad media → utiliza la CPU
- Prioridad baja → posee el mutex, pero no se ejecuta
- **Se puede mitigar mediante herencia de prioridad:**
pthread_mutexattr_t attributes;
pthread_mutexattr_init(&attributes);
pthread_mutexattr_setprotocol(&attributes, **PTHREAD_PRIO_INHERIT**);
pthread_mutex_init(&mutex, &attributes);
pthread_mutexattr_destroy(&attributes);

Secciones críticas demasiado largas

- Nunca debería mantenerse un lock mientras se realizan operaciones potencialmente lentas:

```
pthread_mutex_lock(&mutex);
```

```
/* Evitar aquí operaciones lentas o bloqueantes */
```

```
pthread_mutex_unlock(&mutex);
```

Deben minimizarse:

- llamadas de red;
- acceso a disco;
- esperas;
- asignaciones dinámicas;
- logging intensivo.

Asignación dinámica

- **malloc()** y **free()** no ofrecen necesariamente tiempos de ejecución deterministas.
- En partes críticas puede ser preferible:
 - reservar memoria durante la inicialización;
 - reutilizar buffers;
 - utilizar pools de tamaño fijo;
 - evitar crecimiento dinámico durante la ejecución crítica.

Spinlocks y prioridad

- Un **spinlock** puede consumir toda la CPU mientras espera.
- Si el hilo que posee el lock tiene menor prioridad y no obtiene CPU, la espera puede prolongarse indefinidamente.

Planificación y permisos

- Las políticas habituales de tiempo real en Linux son:
 - **SCHED_FIFO**
 - **SCHED_RR**
- Asignan una prioridad estática entre **1, la menor, y 99, la mayor**. Los hilos de tiempo real tienen preferencia sobre los planificados con políticas normales como SCHED_OTHER
- Debe comprobarse siempre el resultado al configurar prioridades, ya que normalmente se necesitan permisos adecuados.

Planificación y permisos

SCHED_FIFO

– FIFO → *First In, First Out*.

Un hilo continúa ejecutándose hasta que:

- termina;
- se bloquea;
- cede voluntariamente la CPU;
- aparece otro hilo ejecutable con mayor prioridad.

– No utiliza un cuanto temporal para alternar hilos de la misma prioridad.

Hilo A, prioridad 50

Hilo B, prioridad 50

A puede continuar hasta bloquearse o ceder la CPU.

B espera.

Planificación y permisos

- **SCHED_RR**

- RR significa *Round Robin*.
- Funciona de manera parecida a FIFO, pero los hilos de la misma prioridad se alternan mediante un cuanto temporal:
- Hilo A, prioridad 50
- Hilo B, prioridad 50
- A -> B -> A -> B

Planificación y permisos: comando chrt

- Consultar configuración:
- Proceso actual: **chrt --pid \$\$**
- Proceso determinado: **chrt --pid un_pid**
- Prioridades admitidas: **chrt --max**
- Mostrar política y prioridad de procesos e hilos:
ps -eLo pid,tid,cls,rtprio,pri,comm
- **Clases habituales:**
 - **TS** = planificación normal
 - **FF** = SCHED_FIFO
 - **RR** = SCHED_RR

chrt permite consultar o modificar los atributos de planificación de un proceso.

Ejecución con tiempo real

- Ejecutar con **SCHED_FIFO**
 - `sudo chrt --fifo 50 ./programa`
 - `sudo chrt -f 50 ./programa`
- Ejecutar con **SCHED_RR**
 - `sudo chrt --rr 50 ./programa`
 - `sudo chrt -r 50 ./programa`
- Volver a la **planificación normal**:
 - `sudo chrt --other --pid 0 un_pid`

Instalar

- Comprobar si esta en el sistema:
which chrt
- Si no aparece:
sudo apt update
sudo apt install util-linux
- Prueba:
sudo chrt -r 20 sleep 10
- Localizar el proceso mientras se ejecuta
pgrep sleep
- Consultar su política:
chrt -p un_pid
- Se debería de ver:
SCHED_RR
Prioridad: 20

Precauciones

- Evitar prioridades muy altas y bucles infinitos con SCHED_FIFO
- Porque podrían dejar el escritorio Ubuntu / Lubuntu sin respuesta.
- Utiliza prioridades moderadas y programas de duración limitada.

Problemas clásicos

- Race conditions – Detección y mitigación
- Deadlocks y livelocks – estrategias de evitación
- Productor / Consumidor – implementación completa

Race conditions

- Una race condition ocurre cuando el resultado depende del orden impredecible en que se ejecutan varios hilos.
- Existe una race condition cuando dos hilos acceden concurrentemente a la misma posición de memoria y, al menos, uno escribe

Race conditions

- Por ejemplo: un contador compartido por varios hilos y sin ningún mecanismo de sincronización.

```
static int counter = 0;
```

- Si dos o más hilos ejecutan:

```
counter++;
```

- Detección y mitigación

Race conditions

- La expresión no es necesariamente atómica
- 1. Leer counter
- 2. Incrementar la copia
- 3. Escribir el resultado

- Counter inicial: 10
- Hilo A lee 10
- Hilo B lee 10
- Hilo A escribe 11
- Hilo B escribe 11
- **Resultado: 11**
- **Esperado: 12**

Detección con ThreadSanitizer

- ThreadSanitizer, **TSan**, instrumenta el programa para detectar carreras de datos.
- Debe activarse tanto durante la compilación como durante el enlazado.

```
gcc -std=c11 -g -O1 \  
    -fsanitize=thread \  
    race.c -o race_tsan \  
    -pthread
```

Ejecución: `./race_tsan`

El informe suele identificar:

- posición de memoria compartida;
- hilos implicados;
- línea de cada acceso conflictivo;
- lugar donde se crearon los hilos.

TSan incrementa considerablemente el tiempo de ejecución y el consumo de memoria, por lo que se usa en desarrollo y pruebas.

Detección con Helgrind

- **Helgrind** forma parte de **Valgrind** y detecta carreras, uso incorrecto de la API pthread y posibles deadlocks por orden inconsistente de locks.

```
gcc -std=c11 -g -O0 \  
    race.c -o race \  
    -pthread
```

```
valgrind --tool=helgrind ./race
```

Mitigación con mutex

```
static int counter = 0;
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *argument)
{
    (void)argument;
    for (int i = 0; i < INCREMENTS; ++i) {
        pthread_mutex_lock(&mutex);
        counter++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

Sólo un hilo puede ejecutar la sección crítica simultáneamente:

lock

modificar dato compartido

unlock

Optimización

- **Cada hilo puede acumular localmente:**

```
static void *worker(void *argument)
{
    (void)argument;
    int local_counter = 0;
    for (int i = 0; i < INCREMENTS; ++i) {
        local_counter++;
    }
    pthread_mutex_lock(&mutex);
    counter += local_counter;
    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

Mitigación con atomics

- Se pueden utilizar en **C11**
 - `#include <stdatomic.h>`
 - `static atomic_int counter = 0;`
- **Actualización:**
 - `atomic_fetch_add(&counter, 1);`

```
static void *worker(void *argument)
{
    (void)argument;
    for (int i = 0; i * INCREMENTS; ++i) {
        atomic_fetch_add(&counter, 1);
    }
    return NULL;
}
```

Los **atomics** son adecuados para contadores, flags y estados sencillos.

Para proteger varias operaciones relacionadas o **estructuras complejas**, normalmente es preferible un **mutex**.

Estrategias

- Dividir los datos para que cada hilo modifique regiones diferentes.
- Minimizar el estado compartido.
- Mantener secciones críticas cortas.
- Definir claramente qué lock protege cada dato.
- No usar volatile como sustituto de mutexes o atomics.
- Ejecutar pruebas repetidas con diferentes cargas y número de hilos.

Resumen

Problema	Mitigación habitual
Contador compartido	Atomic o mutex
Estructura compartida	Mutex
Muchos lectores	RWLock
Cola productor/consumidor	Mutex y condvars
Regiones independientes	Partición de datos
Detección de carreras	TSan o Helgrind

Deadlocks y livelocks

Ambos impiden que el programa progrese:

- **Deadlock:** los hilos quedan bloqueados esperando recursos.
- **Livelock:** los hilos siguen ejecutándose, pero reaccionan entre sí sin realizar trabajo útil.

Deadlocks

- Un deadlock ocurre cuando varios hilos esperan indefinidamente recursos retenidos por otros hilos.
- Ejemplo:
 - Hilo A posee mutex_a y espera mutex_b
 - Hilo B posee mutex_b y espera mutex_a

Deadlocks

Normalmente aparecen estas cuatro condiciones:

- 1.Exclusión mutua:** el recurso solo puede usarlo un hilo.
- 2.Retención y espera:** un hilo conserva un recurso mientras espera otro.
- 3.Sin expropiación:** no se puede retirar forzosamente el recurso.
- 4.Espera circular:** cada hilo espera un recurso retenido por otro.

Romper cualquiera de estas condiciones evita el deadlock.⁹³

Deadlock - mitigación

- Todos los hilos deben adquirir los mutex en el mismo orden:

```
pthread_mutex_lock(&mutex_a);  
pthread_mutex_lock(&mutex_b);  
    /* Sección crítica */  
pthread_mutex_unlock(&mutex_b);  
pthread_mutex_unlock(&mutex_a);
```

Buenas prácticas

- Reducir el número de locks simultáneos
- Intento sin bloqueo
- Espera con límite temporal

Reducir el número de locks simultáneos

- Siempre que sea posible, liberar un mutex antes de adquirir otro:

```
pthread_mutex_lock(&mutex_a);  
/* Operación protegida por mutex_a */  
pthread_mutex_unlock(&mutex_a);  
  
pthread_mutex_lock(&mutex_b);  
/* Operación protegida por mutex_b */  
pthread_mutex_unlock(&mutex_b);
```

Intento sin bloqueo

- **pthread_mutex_trylock()** vuelve inmediatamente si el mutex está ocupado:

```
pthread_mutex_lock(&mutex_a);  
int result = pthread_mutex_trylock(&mutex_b);  
if (result != 0) {  
    pthread_mutex_unlock(&mutex_a);  
    /* Esperar o reintentar posteriormente */  
} else {  
    /* Se poseen ambos mutexes */  
    pthread_mutex_unlock(&mutex_b);  
    pthread_mutex_unlock(&mutex_a);  
}
```

Este patrón puede evitar el bloqueo permanente, pero una estrategia de reintento incorrecta puede producir un livelock.

Espera con límite temporal

```
struct timespec limit;  
/* Calcular instante límite */  
int result = pthread_mutex_timedlock( &mutex, &limit);  
  
if (result == ETIMEDOUT) {  
    /* Liberar recursos o cancelar la operación */  
}
```

El timeout evita quedarse bloqueado eternamente, pero no corrige por sí mismo el diseño incorrecto.

Detección de deadlocks

- **Helgrind** puede detectar posibles deadlocks causados por órdenes de adquisición inconsistentes.
 - **gcc** -std=c11 -g -O0 deadlock.c -o deadlock -pthread
 - valgrind --tool=**helgrind** ./deadlock
- Puede mostrar mensajes del tipo:
 - ***lock order violated***
 - Indicando que el programa ha usado diferentes órdenes para adquirir los mismos locks.

Livelocks

- En un **livelock** los hilos no están bloqueados.
- Continúan ejecutándose, pero se impiden mutuamente progresar.
- Ejemplo:
 - Hilo A detecta conflicto y libera sus recursos
 - Hilo B detecta conflicto y libera sus recursos
 - Hilo A vuelve a intentarlo
 - Hilo B vuelve a intentarlo
 - El ciclo se repite permanentemente

Ejemplo

```
static void lock_both(void){
    while (1) {
        pthread_mutex_lock(&mutex_a);
        if (pthread_mutex_trylock(&mutex_b) == 0) {
            break;
        }
        pthread_mutex_unlock(&mutex_a);
    }
    /* Se poseen ambos mutexes */
    pthread_mutex_unlock(&mutex_b);
    pthread_mutex_unlock(&mutex_a);
}
```

Mitigación del livelock

- **Espera aleatoria o backoff**
 - En lugar de reintentar inmediatamente:

```
pthread_mutex_unlock(&mutex_a);  
struct timespec delay = {  
    .tv_sec = 0,  
    .tv_nsec = 1000000  
};  
nanosleep(&delay, NULL);
```

En sistemas reales puede usarse un retraso variable para evitar que los hilos vuelvan a coincidir continuamente.

Mitigación del livelock

- **Número máximo de reintentos**

```
#define MAX_RETRIES 10
for (int attempt = 0;
    attempt < MAX_RETRIES;
    ++attempt) {
    /* Intentar adquirir recursos */
}
```

Si se supera el límite:

- se cancela la operación;
- se devuelve un error;
- se reprograma el trabajo.

Mitigación del livelock

- **Orden fijo de locks**
 - La solución preferible sigue siendo establecer un orden global:
 - mutex_a -> mutex_b -> mutex_c
 - Esto evita depender de reintentos.

Mitigación del livelock

- **Coordinación centralizada**
 - Otra opción es que un mutex único proteja la adquisición de varios recursos:
 - `pthread_mutex_lock(&coordination_mutex);`
 - **`/* Adquirir o modificar los recursos */`**
 - `pthread_mutex_unlock(&coordination_mutex);`

Detección de livelocks

- Los livelocks son más difíciles de detectar automáticamente porque los hilos continúan ejecutándose.
- **Indicadores habituales:**
 - uso elevado de CPU;
 - muchas llamadas repetidas a `trylock()`;
 - ausencia de progreso;
 - contadores de trabajo que no avanzan;
 - logs repetitivos;
 - timeouts continuos.

Herramientas

- **top -H -p PID**
 - Mostrar el consumo de CPU de cada hilo
- **strace -f -p PID**
 - Permite observar las llamadas al S.O.

Productor / Consumidor

- **Implementación completa**
 - Ya implementado con **condvars** y **mutex** o con **semáforos** y **mutex**